

Preparação do Sistema para Operação

Trabalho Prático 5

Aluno: Guilherme China

Projeto: Sistema de Biblioteca

Objetivo

Preparar os microsserviços para operação por meio de containerização, orquestração, monitoramento, automação de entrega e testes.

1. Visão geral da solução

O sistema é formado pelo frontend React, pela API principal de biblioteca e pelo serviço de notificações. A API mantém livros, leitores e empréstimos. O serviço de notificações consome pelo RabbitMQ os eventos de empréstimo e devolução. Cada serviço utiliza seu próprio banco PostgreSQL.

```
Usuário
|
Frontend React + Nginx
|
API Biblioteca ---- PostgreSQL Biblioteca
|
RabbitMQ ---- Serviço de Notificações ---- PostgreSQL Notificações

Prometheus ---- métricas
Promtail + Loki ---- logs
OpenTelemetry + Tempo ---- traces
Grafana ---- consulta unificada
```

O frontend usa o Nginx como servidor dos arquivos estáticos e como proxy para as rotas da API. A integração de escrita entre os dois microsserviços permanece assíncrona, com Transactional Outbox no produtor e consumo idempotente no destinatário.

Principais diretórios

- backend e notificacoes-service: aplicações Spring Boot;
- frontend: aplicação React e configuração do Nginx;
- k8s: recursos de implantação e escalabilidade;
- monitoring: configurações de métricas, logs e traces;
- .github/workflows: automação de CI/CD.

2. Containerização com Docker

Os três componentes da aplicação possuem Dockerfiles com build em múltiplos estágios. Os artefatos são compilados em imagens de construção e copiados para imagens finais menores. Backend, notificações e frontend executam com usuários sem privilégios e possuem health checks.

O Docker Compose inicializa as aplicações, dois bancos PostgreSQL, RabbitMQ, Prometheus, Loki, Promtail, Tempo e Grafana. Redes separadas organizam a comunicação da aplicação e da observabilidade, enquanto volumes preservam os dados.

```
docker compose up -d --build
docker compose ps
./scripts/smoke-test.sh
```

Inserir captura do comando `docker compose ps`
com os serviços ativos e saudáveis.

3. Orquestração com Kubernetes

Os manifestos são aplicados com Kustomize. PostgreSQL e RabbitMQ usam StatefulSets e armazenamento persistente. Backend, notificações e frontend usam Deployments e Services. As aplicações possuem duas réplicas iniciais, probes de liveness e readiness e limites de CPU e memória.

Backend e notificações possuem HorizontalPodAutoscalers com mínimo de duas e máximo de cinco réplicas. O alvo é 70% de utilização média de CPU. PodDisruptionBudgets preservam pelo menos uma réplica disponível durante interrupções voluntárias.

```
kubectl apply -k k8s
kubectl -n biblioteca get pods
kubectl -n biblioteca get deployments,statefulsets
kubectl -n biblioteca get hpa,pdb
```

Inserir captura do Kubernetes mostrando
pods Running, réplicas e HPAs.

4. Monitoramento e rastreamento

O Spring Boot Actuator expõe health checks e métricas para o Prometheus. O Promtail coleta os logs dos contêineres e pods e os envia ao Loki. Micrometer e OpenTelemetry geram traces e os exportam por OTLP para o Tempo.

O Grafana possui os três datasources provisionados. Dessa forma, métricas, logs e traces podem ser consultados na mesma interface. Os logs incluem traceId e spanId, permitindo abrir no Tempo a transação relacionada a uma linha do Loki.

Fonte	Consulta de demonstração
Prometheus	up ou http_server_requests_seconds_count
Loki	{service=~"backend notificacoes-service"}
Tempo	resource.service.name = biblioteca

Inserir captura do Grafana Explore
com logs correlacionados ou um trace aberto.

5. Testes abrangentes

Os testes automatizados cobrem regras de negócio, persistência, auditoria, endpoints HTTP, criação de eventos na Outbox e consumo idempotente no serviço de notificações. Foram implementados testes unitários e testes com o contexto web e de persistência do Spring.

Além deles, o script de smoke test verifica os componentes em conjunto. Ele cria leitor e livro, registra um empréstimo, aguarda a notificação assíncrona, registra a devolução, aguarda o segundo evento e consulta as métricas dos dois serviços.

```
cd backend && mvn verify
cd ../notificacoes-service && mvn verify
cd ..
./scripts/smoke-test.sh
```

O cenário integrado também foi executado na implantação Kubernetes, passando pelo frontend, API, PostgreSQL, RabbitMQ e serviço de notificações.

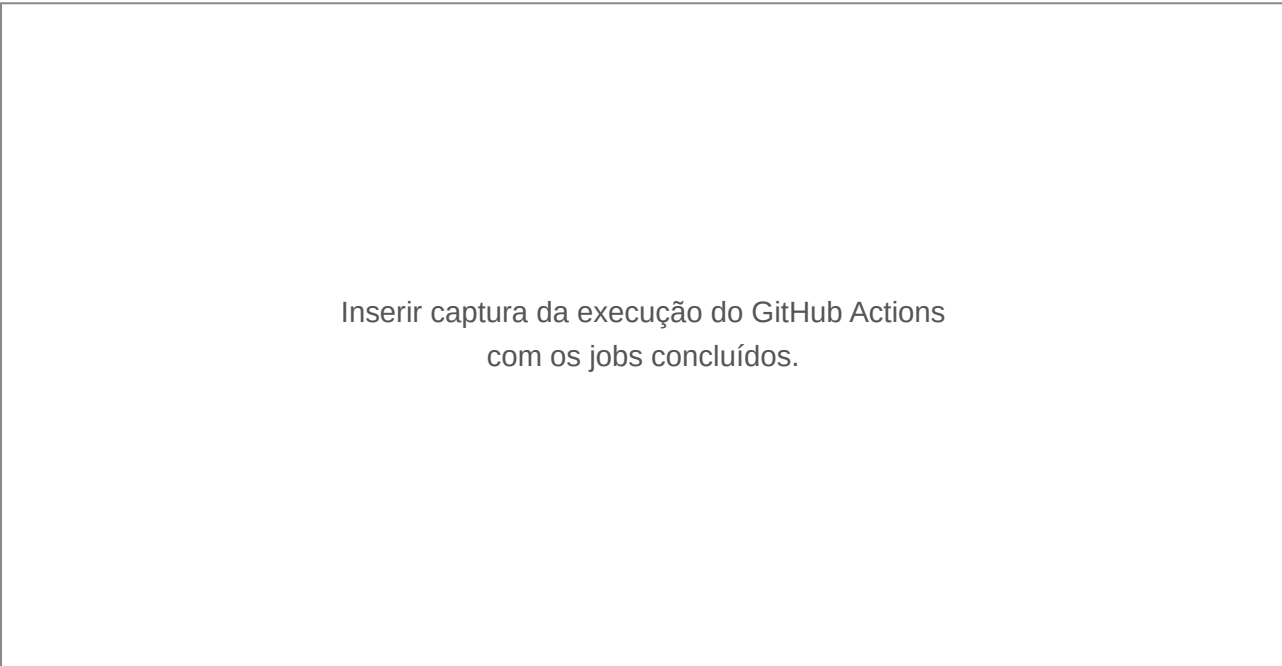
Inserir captura dos testes Maven e da mensagem
"Fluxo ponta a ponta concluído".

6. Integração e entrega contínuas

O GitHub Actions executa o workflow em pushes e pull requests para as branches main e master. A automação testa os dois microsserviços, gera o build do frontend, renderiza os manifestos Kubernetes e executa o teste integrado com Docker Compose.

Depois da aprovação dos testes, o workflow constrói e publica três imagens no GitHub Container Registry. Cada imagem recebe uma tag ligada ao commit e a branch principal também publica a tag latest.

O deploy Kubernetes é opcional. Quando a variável `ENABLE_KUBERNETES_DEPLOY` e o secret `KUBE_CONFIG` são configurados, a pipeline aplica os manifestos, troca as imagens pelas versões publicadas e aguarda o rollout dos três componentes.



Inserir captura da execução do GitHub Actions
com os jobs concluídos.

7. Operação e conclusão

A aplicação pode ser reproduzida localmente com Docker Compose e implantada em um cluster Kubernetes com os mesmos limites de responsabilidade entre os serviços. Health checks e probes detectam indisponibilidade, enquanto réplicas, HPA e PDB auxiliam na continuidade da operação.

A combinação de Prometheus, Loki, Tempo e Grafana permite observar disponibilidade, consumo de recursos, erros e o caminho de cada requisição. A pipeline reduz etapas manuais e impede a publicação das imagens quando os testes falham.

Itens entregues

- Dockerfiles e Docker Compose do ambiente completo;
- manifestos Kubernetes com implantação e escalabilidade;
- configuração de métricas, logs e traces;
- workflow de CI/CD pronto para um novo repositório GitHub;
- testes automatizados e cenário integrado;
- documentação de execução e roteiro da apresentação.

O roteiro de vídeo apresenta os comandos, as falas e a ordem dos fluxos práticos de Docker, aplicação, monitoramento, Kubernetes, testes e GitHub Actions.